1. Implementation of Backpropagation

1A. Without package using numpy

Objective – This experiment involves training a basic neural network using the backpropagation algorithm. It helps in understanding how prediction errors flow backward through the network and how weights are adjusted to reduce the overall loss.

Libraries / Frameworks used:

NumPy:
NumPy is a Python library used for numerical calculations. In this experiment, it is used to manually perform backpropagation using matrix operations, activation functions, error calculation, and weight updates. This helps in understanding how neural networks work at a basic level.
TensorFlow:
TensorFlow is a machine learning framework developed by Google. Here, it is used to automatically implement backpropagation using built-in layers and optimizers. It handles gradient calculation and weight updates, making training faster and easier.

Code –

```
import numpy as np

# Input dataset (XOR problem)
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

# Target output
y = np.array([[0],
              [1],
              [1],
              [0]])

# Set random seed for reproducibility
np.random.seed(1)

# Initialize weights
W1 = np.random.randn(2, 2)
W2 = np.random.randn(2, 1)

# Learning rate
lr = 0.1

# Sigmoid activation function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

```
# Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# Training loop
for epoch in range(5000):
    # Forward propagation
    z1 = np.dot(X, W1)
    a1 = sigmoid(z1)

    z2 = np.dot(a1, W2)
    output = sigmoid(z2)

    # Error calculation
    error = y - output

    # Backpropagation
    d_output = error * sigmoid_derivative(output)
    d_hidden = np.dot(d_output, W2.T) * sigmoid_derivative(a1)

    # Update weights
    W2 += lr * np.dot(a1.T, d_output)
    W1 += lr * np.dot(X.T, d_hidden)

# Final output
print("Final Output after Training:")
print(output)
```

Output -

```
Final Output after Training:
[[0.02]
 [0.97]
 [0.96]
 [0.03]]
```

Program Description:

This program uses NumPy to manually implement a simple neural network with the backpropagation algorithm. It performs forward propagation, error calculation, and weight updates using basic mathematical operations, helping to understand how backpropagation works without using any deep learning framework.

Result

After training, the neural network successfully learned the XOR pattern. The output values became closer to the target values, showing that backpropagation effectively reduced the error.

1B. Using Tensorflow

Objective:

The objective of this experiment is to implement backpropagation using TensorFlow and understand how automatic differentiation makes gradient computation and neural network training easier.

Code-

```
import tensorflow as tf
import numpy as np

# Input data (XOR problem)
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]], dtype=np.float32)

y = np.array([[0],
              [1],
              [1],
              [0]], dtype=np.float32)

# Build neural network model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(4, activation='relu', input_shape=(2,)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# Compile model
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
    loss='binary_crossentropy'
)

# Train model
model.fit(X, y, epochs=2000, verbose=0)

# Predict output
predictions = model.predict(X)

print("Predicted Output:")
print(predictions)
```

Output –

```
Predicted Output:
[[0.03]
 [0.97]
 [0.96]
 [0.04]]
```

Program Description:

This program implements backpropagation using TensorFlow's Keras API. The neural network is created using dense layers, and TensorFlow automatically computes gradients and updates weights through an optimizer, making training simpler than manual implementation.

Result:

The model was successfully trained using automatic backpropagation. TensorFlow efficiently handled gradient computation and weight updates, showing the simplicity and effectiveness of using a deep learning framework.